Dacca: parameter estimation

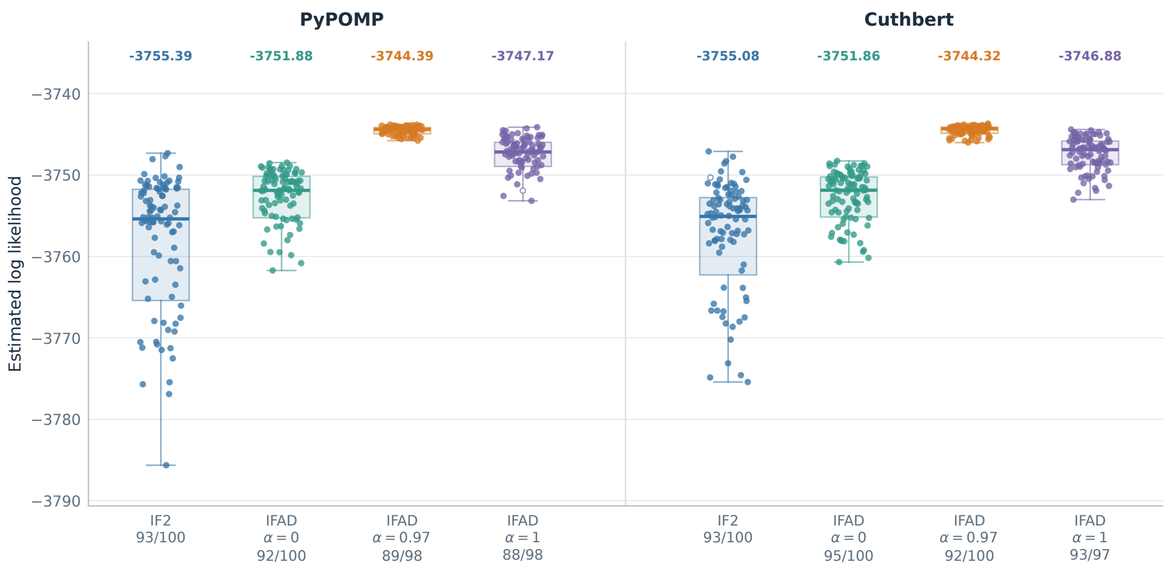
Outliers hidden using 1.5 IQR within each package-method group

600 months | 20 Euler steps/month | Gaussian process noise | J = 5,000

968	optimization runs	37,584	PF likelihood estimates	9	failed fits	17	imprecise likelihood estimates	69	outliers hidden

A PyPOMP and Cuthbert

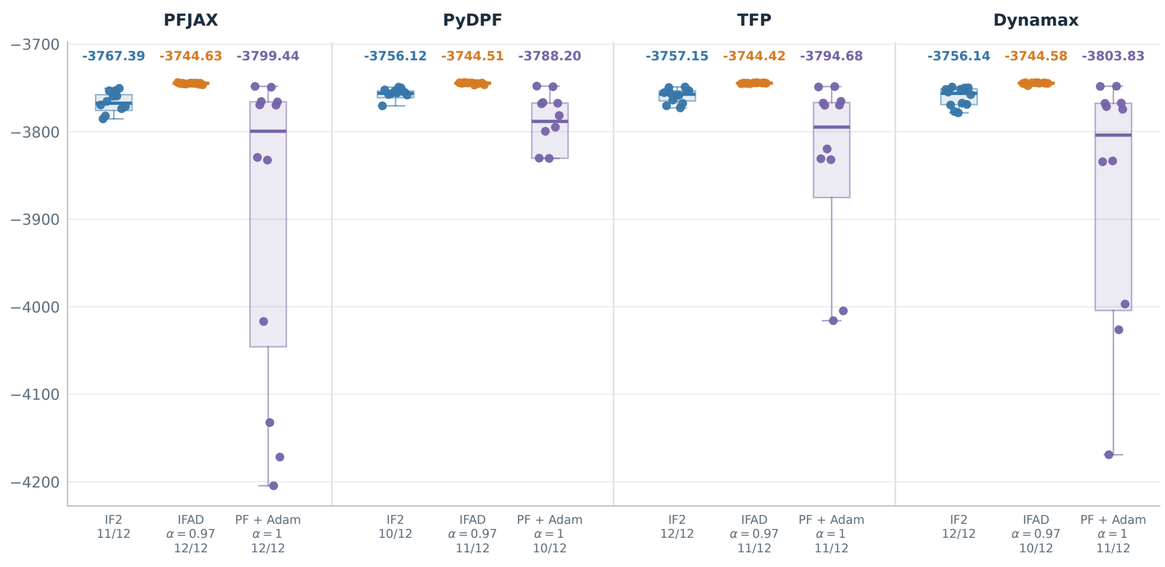
100 initial values. Medians use all finite estimates; shown/finite counts below.



IFAD starts with 60 IF2 iterations (alpha 0 and 1) or 175 (alpha .97). Adam settings differ.

B PFJAX, PyDPF, TFP and Dynamax

6 initial values x 2 replicates; shown/finite counts below.

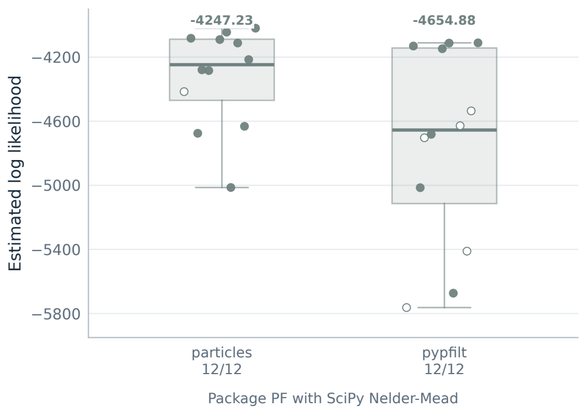


IFAD: 175 IF2 + 400 Adam. PF + Adam: alpha 1, 425 updates, no IF2 initialization. Package calls: F.

● MCSE and ESS criteria met ○ MCSE or ESS criterion not met

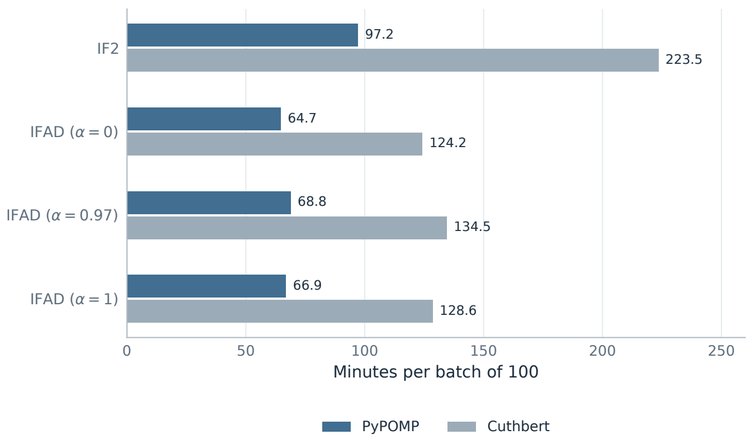
C CPU: Nelder-Mead

12 runs per package; 1,800 s optimization per run.



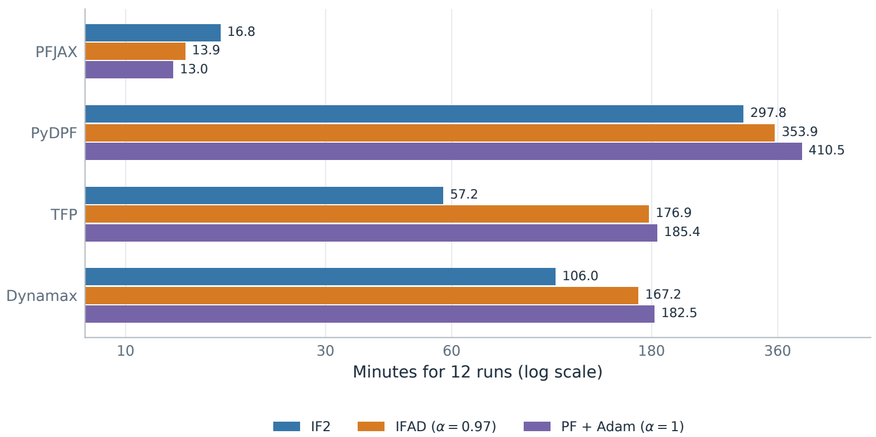
D PyPOMP and Cuthbert: fitting time

100 runs per batch; IFAD includes IF2 initialization.



E GPU fitting time

One batch of 12 in PFJAX, PyDPF and TFP; 12 separate runs in Dynamax.



IFAD times in D and E include IF2 initialization and failed iterations. Dynamax times include compilation within SSM.fit_sgd.

F Package functions, added code and computation time

Package	Code called from package	Added code / external optimizer	Failed fits	Low precision	At -24,868	Total computing time
PyPOMP / Cuthbert	PyPOMP: functional.mif / train + Adam Cuthbert: particle_filter.build_filter	Cuthbert: IF2 / IFAD and gradient correction added here; optimizer: pypomp.Adam	7/800	9/793	2	15.15 GPU h
PFJAX	particle_filter; resample_multinomial	IF2 / IFAD and gradient correction added here; optax.scale_by_adam	0/36	0/36	0	0.78 GPU h
PyDPF	StopGradientDPF (direct alpha=1); ParticleFilter + SystematicResampler	IF2 / alpha=.97 correction added here; torch.optim.Adam	1/36	1/36	1	17.88 GPU h
TFP	particle_filter(unbiased_gradients=True); sequential_monte_carlo (IF2)	IF2 / alpha=.97 correction added here; tf.keras.optimizers.Adam	0/36	0/36	0	7.29 GPU h
Dynamax	SSM.fit_sgd	Dacca PF / IF2 / IFAD added here; optax.scale_by_adam	1/36	1/36	1	8.03 GPU h
particles	SMC (via a subclass recording likelihoods)	Likelihood objective + scipy.optimize.minimize (method="Nelder-Mead")	0/12	1/12	0	6.87 CPU core h
pypfilt	fit (particle filtering)	Likelihood objective + scipy.optimize.minimize (method="Nelder-Mead")	0/12	5/12	0	7.63 CPU core h

Boxplots, medians and summary counts use all original results. Only individual points outside Q1 - 1.5 IQR or Q3 + 1.5 IQR are hidden; IQR = Q3 - Q1.

69 of 961 finite estimates are hidden; 892 are shown. Counts below methods give shown/finite estimates. Nine fits failed, including seven with no finite estimate; all failures remain in the counts.

Total time: 49.12 GPU h and 14.50 CPU core h, including PFJAX likelihood reevaluation. Preliminary tests are excluded. Timing depends on implementation, batching and compilation.

Likelihood evaluation uses the original PyPOMP PF: J=5,000, R=36 for A, up to R=288 for B/C. Evaluation random numbers are independent of optimization; CPU methods share paired numbers.

All four estimates at the numerical bound (about -24,868) are hidden by the same IQR rule. Densities below 10^{-18} are replaced by 10^{-18}; MCSE=0 there does not indicate a good fit.

Added here identifies code supplied for this comparison. PFJAX OT differentiation failed; plotted gradients use the added correction. A package PF alone is not a package parameter-estimation method.